Tema 3

Generalidades

3.1. Generalidades

Los sockets constituyen la abstracción fundamental para la comunicación entre procesos en redes de computadores. Desde el punto de vista del modelo TCP/IP, un socket es un extremo de comunicación que combina una dirección IP y un número de puerto, permitiendo identificar de manera única un proceso dentro de un host. En aplicaciones distribuidas, los sockets son el mecanismo base sobre el cual se construyen protocolos de más alto nivel, servicios web, middleware y arquitecturas orientadas a servicios. (Tanenbaum & Van Steen, 2017)

Formalmente, un socket puede definirse como una interfaz de programación (API) que permite a un proceso enviar y recibir datos a través de una red utilizando protocolos de transporte como TCP o UDP. Esta interfaz estandarizada fue popularizada en sistemas Unix bajo la especificación BSD Sockets y posteriormente adoptada por la mayoría de sistemas operativos modernos.

En términos conceptuales, los sockets permiten implementar el paradigma cliente-servidor, donde:

- El servidor crea un socket, lo asocia a un puerto y escucha conexiones entrantes.
- El cliente crea un socket y solicita conexión al servidor.
- Una vez establecida la conexión, se produce intercambio bidireccional de datos.

La programación con sockets surge de la necesidad de establecer comunicación entre procesos que se ejecutan en máquinas diferentes. A diferencia de la comunicación local (IPC), en redes se requiere:

- Identificación de nodos (direcciones IP)
- Identificación de procesos (puertos)
- Protocolos de transporte confiables o no confiables
- Manejo de errores de red

Componentes de un socket

Un socket se identifica mediante: IP origen, Puerto origen, IP destino, Puerto destino, Protocolo. Esta tupla garantiza unicidad de la conexión.

Tipos principales de sockets

1. **Sockets orientados a conexión (TCP)**

- Protocolo confiable
- Garantiza entrega ordenada
- Control de congestión
- Confirmación de recepción

1. **Sockets no orientados a conexión (UDP)**

- No garantizan entrega.
- Menor latencia.
- Adecuado para transmisión en tiempo real.

Ciclo de vida de un socket TCP

Servidor

1. socket()
2. bind()
3. listen()
4. accept()
5. read()/write()
6. close()

Cliente

1. socket()
2. connect()
3. read()/write()
4. close()

Este flujo refleja el establecimiento de conexión mediante el three-way handshake del protocolo TCP.

3.2. Comunicación en aplicaciones distribuidas

La comunicación es el núcleo de toda aplicación distribuida. A diferencia de sistemas monolíticos, los componentes distribuidos deben intercambiar mensajes explícitamente. Según Coulouris et al. (2012), la comunicación distribuida puede clasificarse en:

- **Comunicación síncrona:** el emisor espera respuesta.
- **Comunicación asíncrona**: el emisor continúa sin bloquearse.
- **Unicast:** entre dos nodos.
- **Multicast:** a múltiples nodos simultáneamente.

3.2.1. Modelos de comunicación

1. **Paso de mensajes (Message Passing)**

Es el modelo fundamental en sistemas distribuidos. Los procesos envían mensajes estructurados a través de sockets.

1. **RPC (Remote Procedure Call)**

Permite invocar procedimientos remotos como si fueran locales.

1. **RMI (Remote Method Invocation)**

Extiende RPC a entornos orientados a objetos.

1. **Mensajería basada en colas**

Utiliza intermediarios (brokers) para desacoplar emisores y receptores.

3.2.2. Serialización de datos

Para transmitir datos complejos, estos deben convertirse a formato binario o textual (serialización).

Ejemplos

- JSON
- XML
- Protocol Buffers

La serialización influye en rendimiento, interoperabilidad y consumo de ancho de banda.

3.2.3. Problemas de comunicación distribuida

1. Latencia variable
2. Pérdida de paquetes
3. Duplicación de mensajes
4. Fallos de nodo
5. Ataques de red

Por ello, los sistemas robustos implementan:

- Reintentos automáticos
- Timeouts
- Confirmaciones (ACK)
- Cifrado (TLS)

3.3. Protocolos de la capa de aplicación

Los protocolos de la capa de aplicación definen las reglas para el intercambio de datos entre aplicaciones distribuidas. Operan sobre protocolos de transporte como TCP o UDP.

Modelo OSI y TCP/IP

En el modelo TCP/IP, la capa de aplicación engloba funciones de las capas superiores del modelo OSI. Aquí se definen formatos de mensajes, semántica y comportamiento.

3.3.1. Protocolos principales

1. **HTTP (Hypertext Transfer Protocol)**

Protocolo sin estado ampliamente usado en la Web. Opera sobre TCP. Es la base de API REST.

1. **HTTPS**

Versión segura de HTTP que utiliza TLS para cifrado.

1. **FTP (File Transfer Protocol)**

Transferencia de archivos

1. **SMTP (Simple Mail Transfer Protocol)**

Envío de correo electrónico

1. **DNS (Domain Name System)**

Resolución de nombres de dominio a direcciones IP

1. **WebSocket**

Comunicación bidireccional persistente sobre TCP

3.3.2. REST y arquitecturas orientadas a recursos

REST (Representational State Transfer) es un estilo arquitectónico que utiliza HTTP como protocolo base, aprovechando métodos como:

- GET
- POST
- PUT
- DELETE

REST enfatiza:

- Statelessness
- Identificación uniforme de recursos
- Representaciones intercambiables (JSON, XML)

3.3.3. gRPC y protocolos binarios

gRPC utiliza HTTP/2 y Protocol Buffers, ofreciendo:

- Mayor eficiencia
- Comunicación bidireccional
- Streaming

3.3.4. Seguridad en la capa de aplicación

La seguridad incluye:

- Autenticación (OAuth, JWT)
- Autorización

- Cifrado TLS
- Firmas digitales

En sistemas distribuidos modernos, la elección del protocolo de aplicación impacta directamente en rendimiento, escalabilidad y mantenibilidad.

Recursos complementarios

Documentación oficial de HTTP-IETF

Recurso complementario en línea: https://datatracker.ietf.org/doc/html/rfc9110

https://datatracker.ietf.org/doc/html/rfc9110

Introducción a REST-Roy Fielding (tesis doctoral)

Recurso complementario en línea: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Referencias

Coulouris, G.; Dollimore, J.; Kindberg, T. & Blair, G. (2012). Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.

Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

Tanenbaum, A. & Van Steen, M. (2017). Distributed Systems (3rd ed.).

Autoevaluación

1. Un socket se identifica por:

a. IP y puerto.
b. Solo IP.
c. Solo puerto.
d. CPU.

2. TCP garantiza:

a. Entrega ordenada y confiable.
b. Entrega no confiable.
c. Latencia cero.
d. Ancho de banda infinito.

3. UDP se caracteriza por:

a. Menor sobrecarga y menor confiabilidad.
b. Conexión persistente obligatoria.
c. Garantía de entrega.
d. Uso exclusivo en Web.

4. RPC permite:

a. Invocar procedimientos remotos como locales.
b. Compilar código.
c. Aumentar CPU.
d. Eliminar red.

5. HTTP es un protocolo:

a. De aplicación.
b. De transporte.
c. De red.
d. De enlace.

6. REST se basa principalmente en:

a. HTTP.
b. UDP.
c. FTP.
d. SMTP.

7. DNS convierte:

a. Dominios en direcciones IP.
b. Archivos en paquetes.
c. TCP en UDP.
d. JSON en XML.

8. La serialización sirve para:

a. Convertir datos a formato transmisible.
b. Cifrar datos.
c. Aumentar latencia.
d. Crear sockets.

9. HTTPS agrega a HTTP:

a. TLS.
b. UDP.
c. FTP.
d. JSON.

10. La comunicación asíncrona implica que:

a. El emisor no se bloquea esperando respuesta.
b. El emisor espera respuesta obligatoriamente.
c. No existe red.
d. Solo funciona en UDP.